

QueryQuest – Assignment 1

Aditi
240048

Part I

Q1. DBMS is like a highly intelligent, automated digital assistant managing a massive, interconnected digital warehouse whereas Traditional File System can be thought of as a physical filing cabinet filled with paper folders.

- DBMS is highly structured whereas traditional one is often unstructured.
- In DBMS data is normalized and centralized resulting in high data independence but there is high duplication of data across different files in traditional, results in low data independence.
- In DBMS there is fast and flexible data retrieval whereas custom code must be written to search through files in traditional systems, resulting in slow and rigid data retrieval.
- DBMS uses advanced concurrency control protocols resulting in excellent simultaneous multi-user access. In traditional systems files are locked or overwritten when accessed together, resulting in poor concurrent management.

Scenario:

Imagine PPOC wants to send priority invites for a policy workshop to all CSE students with a CPI above 8.5.

With a DBMS, it takes five seconds. The data lives in one central place, so PPOC just runs a quick query and instantly gets the live, accurate list—even if CSE professors are actively uploading grades at that exact moment.

With a file system, it's a headache. PPOC has to email the CSE department for their student spreadsheet, email the Academic cell for the grade file, and manually match the data line-by-line. If a grade changes in the meantime, PPOC's emailed copy is instantly outdated, leading to wrong invites.

Q2. A strong entity can exist independently of any other entity in the database and has its own primary key. A weak entity cannot exist on its own and depends entirely on a strong entity to

exist. In an ER diagram, a strong entity is drawn with a single rectangle, while a weak entity is distinguished by a double rectangle.

A weak entity's primary key is formed by combining the primary key of its identifying strong entity with its own partial key. And the relationship linking a weak entity to its strong entity is called an *identifying relationship*.

Q3. It should be modeled as a *multivalued attribute*.

A multivalued attribute is used when an entity can have more than one value for that specific attribute at the same time. "a student may have multiple phone numbers" fits this definition perfectly.

A simple attribute can only hold a single atomic value per student. If modeled as simple, could only store one number per student.

A composite attribute is an attribute that is broken down into independent sub-parts. A phone number itself isn't being broken down into parts here; just multiple distinct instances of it are to be stored.

Q4. Super key is set of one or more attributes that can uniquely identify a tuple in a relation. It can contain extra, redundant attributes that aren't strictly necessary for uniqueness. Whereas, Candidate key is a minimal super key that contains no redundant attributes, any attribute removed from it, it loses its ability to uniquely identify the tuple.

{roll_no} and {email} are the candidate keys. Two super keys can be {roll_no, name} (Redundant because roll_no alone is sufficient) and {roll_no, email, name, branch} (The entire schema serves as a super key)

Q5. This will violate the Foreign Key Constraint.

It prevents orphan records and ensures data consistency. It ensures that a relationship cannot exist without a valid target, preventing an enrollment record from pointing to a non-existent student.

The DBMS will completely reject the insert operation and throw an error. No data will be added to the Student table.

Q6. A new *junction table* must be created specifically to represent the relationship.

Its primary key consists of the combination of the primary keys from both participating entity tables, acting as a composite primary key. Each of these components also serves as a foreign key pointing back to its respective original table.

A single row in a relational database table can only store one atomic value per column. If tried to represent a many-to-many relationship as a column inside one of the entity tables, have to store multiple values in a single cell, violating First Normal Form, or duplicate the entire entity row multiple times, which leads to massive data redundancy and inconsistencies.

Part II

Part A

Entity	Key Attribute
Customer	CustomerID
Restaurant	RestaurantID
Order	OrderID
MenuItem	ItemID

- Customer and Order (1:N)
One customer can place multiple orders, but each order belongs to exactly one customer.
- Restaurant and Order (1:N)
One restaurant can fulfill multiple orders, but each order is placed with exactly one restaurant.
- Restaurant and MenuItem (1:N)
One restaurant can offer multiple menu items, but each menu item belongs to that specific restaurant's menu.
- Order and MenuItem (M:N)
An order can contain multiple menu items, and a single menu item can be part of many different orders.

Part B

- Phone_Number (for Customer or Restaurant) qualifies as Multivalued Attribute.
cause a customer or restaurant can have multiple distinct contact numbers active at the same time.

- Customer_Address qualifies as Composite Attribute. cause it can be broken down into smaller, independent sub-attributes like House_No, Street, City, and Pincode for more precise querying.

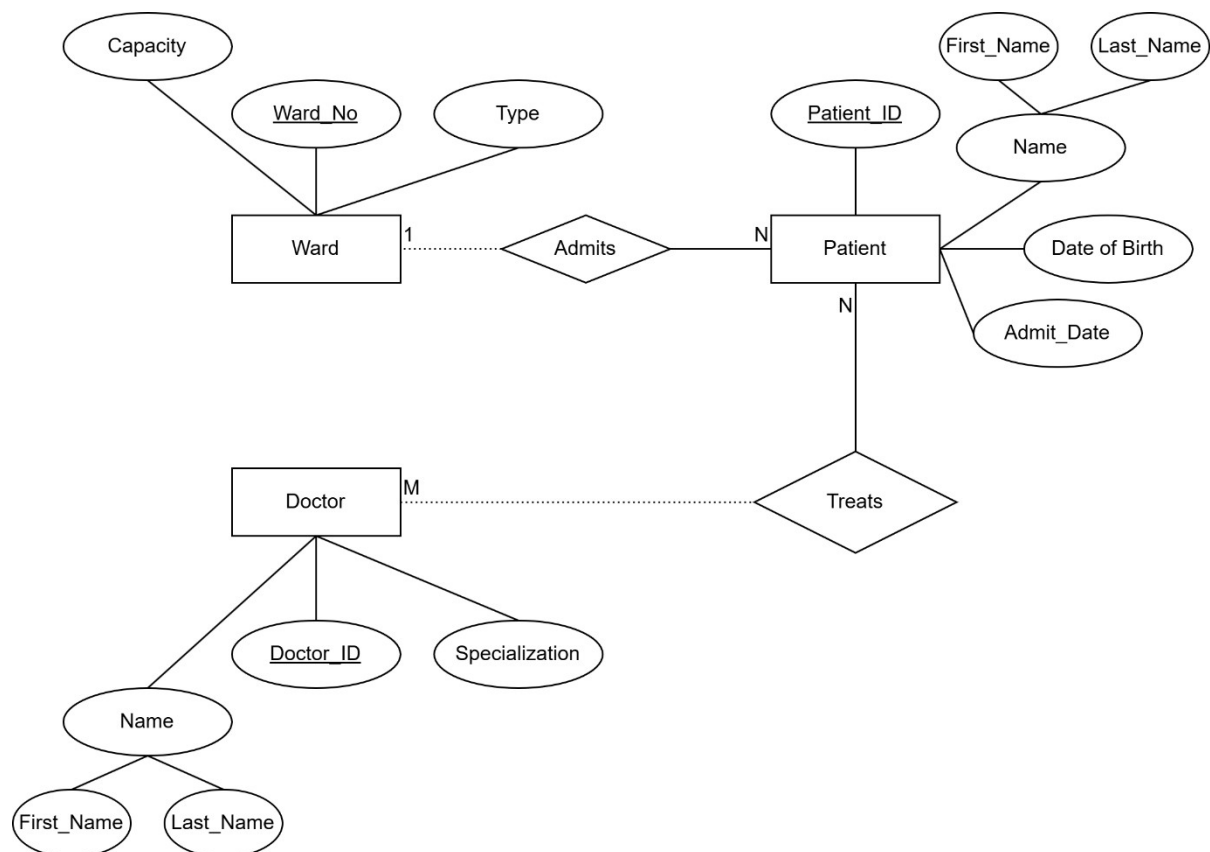
Part C

Order_Items(OrderID, ItemID, Quantity)

- Primary Key: Composite key consisting of (OrderID, ItemID).
- Foreign Keys: * OrderID references Order(OrderID)
ItemID references MenuItem(ItemID)
- Additional Attributes: Quantity (stores the specific amount of that menu item ordered).

Part III

Question 1



Question 2

The Doctor–Patient relationship is $M:N$ because a single doctor can manage multiple patients simultaneously, while a single patient’s treatment plan frequently involves multiple specialists. This many-to-many interaction resolves into an associative entity called Treatment, whose composite primary key is formed by combining the primary keys of both parent entities: (Doctor_ID, Patient_ID).

For participation constraints, the Ward–Patient relationship features total participation on the Patient side because a patient cannot physically reside in the hospital system without being assigned to a specific ward bed. Conversely, the Ward entity has partial participation, as a newly opened ward can exist with zero active patients. Similarly, the Doctor–Patient relationship has total participation for the Patient, ensuring no individual remains unassigned to a medical professional during their stay.

Regarding attribute decisions, the Name attribute for both Doctors and Patients was modeled as a composite attribute breaking down into First_Name and Last_Name. This design choice avoids atomic string errors and allows the hospital administration to cleanly sort records alphabetically or run targeted queries for specific family names.